

شرح Lecture 9 — المراجعة النهائية للكورس

المحاضرة التاسعة مقسمة إلى أربعة أجزاء رئيسية:

1. Recap لكل ما تمت دراسته في المحاضرات السابقة.
2. Beyond Transformer-based LLMs: استخدام الـ Transformer مع الصور والـ Multimodality.
3. Diffusion LLMs: طريقة مختلفة لتوليد النص بدل الـ Autoregressive generation.
4. Closing thoughts: مستقبل المجال، التحديات، والاتجاهات التي يمكن دراستها بعد الكورس.

fall25-cme295-lecture9 

أولاً: الصورة الكاملة للكورس

يمكن تلخيص رحلة الكورس كلها في هذا الخط:



```
Text
↓
Tokenization + Embeddings
↓
Transformer Architecture
↓
Pretraining
↓
Supervised Fine-tuning
↓
Preference Tuning
↓
Reasoning Training
↓
RAG + Tool Calling
↓
```

Agents
↓
Evaluation
↓
Deployment and Optimization

بمعنى أنك بدأت من السؤال:

كيف يفهم المودل الكلمات؟

ثم انتهيت إلى:

كيف أبني نظام AI كاملاً يستطيع التفكير، استدعاء أدوات، البحث في البيانات، تنفيذ خطوات، ثم تقييم أدائه؟

وهذه هي الرسالة الأساسية للكورس:
الـ LLM نفسه مجرد جزء داخل نظام أكبر.

الجزء الأول: مراجعة الكورس

Lecture 1 — Transformers

المحاضرة الأولى بدأت من جملة بسيطة:



A cute teddy bear is reading.

1. Tokenization

أول خطوة هي تقسيم النص إلى وحدات تسمى **Tokens**:



A | cute | teddy bear | is | reading | .

ليس ضروريًا أن يكون كل Token كلمة كاملة؛ فقد يكون:

- كلمة.
- جزءًا من كلمة.
- علامة ترقيم.
- رمزًا برمجيًا.

بعد ذلك يتحول كل Token إلى رقم يسمى `token_id`.

2. Embeddings

المودل لا يفهم الكلمات مباشرة، لذلك تتحول كل كلمة إلى **Vector**:



token_id → embedding vector

مثلاً:



teddy → [0.12, -0.45, 0.81, ...]

الـ Embedding المتعلم يجعل الكلمات المتشابهة دلاليًا قريبة في الـ Vector space.

مثلاً:



teddy bear ≈ toy
teddy bear ≈ soft
teddy bear ≠ database

لكن الـ Embedding وحده لا يكفي؛ لأنه لا يراعي سياق الكلمة.

كلمة مثل:



bank

قد تعني بنكًا ماليًا أو ضفة نهر، وهنا يأتي دور الـ Attention.

3. Query, Key, Value

داخل الـ Self-Attention، يتحول كل Token إلى ثلاثة Vectors:



Query
Key
Value

يمكن فهمهم بهذه الصورة:

- **Query**: ما الذي أبحث عنه؟
- **Key**: ما المعلومات التي أمثلها؟
- **Value**: ما المحتوى الذي سأرسله لو تم اختياري؟

عند تحديث تمثيل كلمة teddy bear مثلاً، يقوم الـ Query الخاص بها بمقارنة نفسه مع الـ Keys الخاصة بكل الكلمات:



q_teddy · k_A
q_teddy · k_cute
q_teddy · k_teddy
q_teddy · k_is
q_teddy · k_reading

كلما كان Dot Product أكبر، كانت العلاقة أقوى.

ثم نحول النتائج إلى احتمالات باستخدام softmax :

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

النتيجة هي Weighted Sum من الـ Values.

بالتالي، تمثيل teddy bear الجديد لم يعد يمثل الكلمة وحدها، بل يمثلها داخل السياق:



cute teddy bear that is reading

4. Multi-Head Attention

بدل وجود Attention واحد، نستخدم عدة Heads.

كل Head قد يتعلم نوعًا مختلفًا من العلاقات:

- Head للعلاقات النحوية.
- Head للعلاقات الدلالية.
- Head لمعرفة الفاعل والفعل.
- Head للعلاقات بعيدة المدى.

ثم نجمع نواتج كل الـ Heads.

5. Transformer Block

كل Transformer Block يحتوي بصورة مبسطة على:



```
graph TD
    Input --> MSA[Multi-Head Self-Attention]
    MSA --> RC1[Residual Connection + Normalization]
    RC1 --> FFN[Feed-Forward Network]
    FFN --> RC2[Residual Connection + Normalization]
    RC2 --> Output
```

Self-Attention

يجعل كل Token يتبادل المعلومات مع Tokens أخرى.

Feed-Forward Network

يعالج كل Token بشكل مستقل بعد حصوله على معلومات السياق.

Residual Connections

تسمح للمعلومات القديمة بالمرور مباشرة:



```
output = input + transformation(input)
```

وهذا يساعد في تدريب مودلات عميقة.

Normalization

تحافظ على استقرار القيم أثناء التدريب.

Lecture 2 — Transformer-based Models and Tricks

بعد فهم البنية الأساسية، انتقل الكورس إلى التحسينات المستخدمة في المودلات الحديثة.

1. Positional Information

الـ Attention نفسه لا يعرف ترتيب الكلمات.

بالنسبة إليه، بدون معلومات إضافية، قد تكون الجملتان متشابهتين:



```
The dog chased the cat.  
The cat chased the dog.
```

لذلك يجب إدخال معلومات المكان.

2. RoPE — Rotary Position Embeddings

بدل إضافة Position Vector عادي إلى الـ Token، يقوم RoPE بتدوير الـ Query والـ Key بزاوية تعتمد على موقع الـ Token.

الفكرة:



```
Token at position m → rotate vector by angle related to m  
Token at position n → rotate vector by angle related to n
```

عند حساب العلاقة بينهما، تظهر المسافة النسبية بين الموقعين.

الميزة المهمة:

RoPE يجعل الـ Attention حساسًا للمسافة النسبية بين الـ Tokens.

ولهذا يُستخدم في عدد كبير من الـ LLMs الحديثة.

3. MHA, MQA, GQA

Multi-Head Attention — MHA

كل Head يمتلك:

 Q head
K head
V head

هذا يعطي مرونة كبيرة، لكنه يستهلك Memory كبيرة، خصوصًا بسبب الـ KV Cache.

Multi-Query Attention — MQA

كل الـ Query Heads تشترك في نفس:

 K
V

الميزة:

- KV Cache أصغر.
- Inference أسرع.

لكن قد يحدث انخفاض في الجودة.

Grouped-Query Attention — GQA

حل وسط:

- توجد عدة Query Heads.
- كل مجموعة منها تشترك في K و V.

 أكبر Memory، أعلى مرونة → MHA
مرونة أقل، أقل Memory → MQA
حل متوازن → GQA

4. Decoder-only Architecture

المودلات الحديثة غالبًا لا تستخدم الـ Encoder-Decoder الكامل الموجود في الـ Transformer الأصلي.

بل تستخدم:



Decoder-only Transformer

مع **Causal Mask**، بحيث لا يستطيع Token رؤية Tokens المستقبلية.

مثلًا عند توقع كلمة بعد:



A teddy bear is

يستطيع المودل رؤية ما قبلها فقط، ولا يستطيع رؤية الإجابة الصحيحة المستقبلية.

Lecture 3 — Large Language Models

1. Autoregressive Language Modeling

المودل يتعلم الاحتمال:

$$P(x_t \mid x_1, x_2, \dots, x_{t-1})$$

أي:

ما احتمال الـ Token التالي بناءً على كل ما سبق؟

عند التوليد:



A
A teddy
A teddy bear
A teddy bear is
A teddy bear is reading

كل Token جديد يعتمد على الـ Tokens السابقة.

2. Mixture of Experts — MoE

داخل الـ Transformer التقليدي يوجد Feed-Forward Network واحد في كل Layer.

في MoE نضع عدة Networks تسمى **Experts**:



Expert 1
Expert 2

Expert 3

...

Expert N

ثم يوجد **Router** أو **Gate** يحدد أي Experts يجب استخدامها لكل Token.

مثلاً:



Token related to code → coding expert

Token related to mathematics → math expert

Token related to language → language expert

ليس من الضروري تشغيل كل Experts، بل عادة يتم اختيار top-k.

مثلاً:



64 total experts

ين فقط Expert يشغل Token لكن كل

الميزة:

زيادة عدد Parameters الكلي دون زيادة Compute بنفس النسبة.

لكن MoE يضيف تحديات مثل:

- Load balancing
- Communication بين الـ GPUs.
- بعض Experts قد تستقبل Tokens أكثر من غيرها.
- Routing instability

3. Response Generation

بعد أن ينتج المودل Probability Distribution للـ Token التالي، يمكن اختيار الـ Token بطرق مختلفة.

Greedy Decoding

اختيار أعلى احتمال دائماً.



$\text{argmax } P(\text{token})$

سريع، لكنه قد يكون متكررًا وغير إبداعي.

Sampling

اختيار Token عشوائيًا وفقًا للتوزيع الاحتمالي.

يعطي تنوعًا أكبر.

Temperature

تتحكم في حدة التوزيع:

- Temperature منخفضة → إجابات أكثر ثباتًا.
- Temperature مرتفعة → إجابات أكثر تنوعًا ومخاطرة.

Top-k

نأخذ أعلى k Tokens فقط، ثم نختار من بينها.

Top-p

نأخذ أصغر مجموعة Tokens يكون مجموع احتمالاتها أكبر من p .

Lecture 4 — LLM Training

1. Scaling Laws

أظهرت الأبحاث أن Loss غالبًا يتحسن بصورة منتظمة عند زيادة:

- عدد الParameters.
- حجم الDataset.
- مقدار الCompute.

لكن لا يكفي تكبير المودل فقط.

يجب توزيع الCompute بصورة مناسبة بين:



Model size

Data size

Training compute

2. Compute-optimal Training

فكرة أبحاث Chinchilla هي أن بعض المودلات القديمة كانت كبيرة جدًا مقارنة بكمية الData التي تدربت عليها.

قد تحصل على مودل أفضل من خلال:



Smaller model + more training tokens

بدلًا من:



Huge model + insufficient tokens

إذن المودل يجب أن يكون Data-compute balanced.

3. FlashAttention

المشكلة في الـ Attention ليست العمليات الحسابية فقط، بل نقل البيانات بين أنواع الـ Memory.

داخل الـ GPU يوجد تقريبًا:

- **HBM**: كبيرة نسبيًا، لكنها أبطأ.
 - **SRAM**: صغيرة، لكنها أسرع وقريبة من Compute Units.
- الـ Attention التقليدي قد ينقل Matrices كبيرة عدة مرات بين الـ HBM و الـ SRAM.
- FlashAttention يقوم بتقسيم الحساب إلى Blocks أو Tiles:



Load small block
Compute attention locally
Keep intermediate results in fast memory
Avoid storing full attention matrix

الميزة:

- Memory أقل.
- سرعة أعلى.
- النتيجة الرياضية نفسها تقريبًا وليست Approximate Attention.

الدرس المهم هنا:

سرعة الخوارزمية لا تعتمد فقط على عدد عمليات الضرب، بل أيضًا على مقدار الـ Memory movement.

4. مراحل تدريب الـ LLM

المحاضرة تلخص التدريب في ثلاث مراحل كبيرة:



Initialized Model
↓
Pretraining
↓
Base Model
↓
Fine-tuning
↓

Task/Instruction Model
↓
Preference Tuning
↓
Aligned Assistant

Pretraining

يتعلم المودل:

- اللغة.
- المعلومات العامة.
- الكود.
- الأنماط.
- العلاقات بين المفاهيم.

الObjective غالبًا:



Next-token prediction

Supervised Fine-Tuning — SFT

نعطي المودل أمثلة Instruction-Response:

```
{  
  "instruction": "Explain gravity",  
  "response": "Gravity is..."  
}
```



JSON

فيتعلم:

- اتباع التعليمات.
- شكل الحوار.
- الإجابة بالطريقة المطلوبة.
- مهام متخصصة.

Preference Tuning

نجعل المودل لا ينتج فقط إجابة صحيحة لغويًا، بل إجابة يفضلها البشر:

- أكثر فائدة.
- أكثر أمانًا.
- أوضح.
- أقل سلوكًا ضارًا.

Lecture 5 — LLM Tuning

Preference Data .1

نعطي المقيمين إجابتين:



Response A
Response B

ثم يختار الإنسان:



A > B

أي أن A مفضلة على B.

الداتا تكون بالشكل:

```
{  
  "prompt": "...",  
  "chosen": "...",  
  "rejected": "..."  
}
```

Bradley-Terry Model .2

نريد تحويل تفضيلات البشر إلى احتمالات.

إذا كان لدينا Reward للإجابة i و Reward للإجابة j :

$$P(y_i > y_j) = \frac{e^{r_i}}{e^{r_i} + e^{r_j}} = \sigma(r_i - r_j)$$

إذا كان:



$r_i \gg r_j$

فاحتمال تفضيل i يصبح قريباً من 1.

Reward Model .3

ندرب مودلاً يأخذ:



Prompt + Response

ويرجع:



Scalar reward

مثلاً:



Good answer → 8.4

Poor answer → 1.2

الـ Reward Model يحاول تقليد تفضيلات البشر.

RLHF .4

في Reinforcement Learning from Human Feedback:



Prompt



Policy LLM generates response



Reward Model scores response



RL updates Policy LLM

المودل الذي يتم تحديثه يسمى:



Policy Model

أما الـ Reward Model فيكون غالباً Frozen أثناء هذه المرحلة.

PPO .5

هدف PPO يتكون بصورة مبسطة من جزأين:



Get high reward

+

Do not move too far from the reference model

لماذا لا نكتفي بزيادة الـ Reward؟

لأن المودل قد يستغل ثغرات الـ Reward Model، وهي ظاهرة تسمى:



Reward hacking

لذلك نضيف KL Penalty تقيس مدى ابتعاد المودل الجديد عن الReference Model.



maximize reward
- $\beta \times \text{KL}(\text{policy} || \text{reference})$

DPO .6

DPO يحاول تحقيق Preference Tuning مباشرة من بيانات:



chosen vs rejected

من دون الحاجة إلى تدريب Reward Model منفصل ثم تشغيل PPO كامل.

الفكرة المبسطة:

زد احتمال الإجابة المختارة وقلل احتمال الإجابة المرفوضة مقارنة بالReference Model.

لذلك DPO:

- أبسط من RLHF التقليدي.
- أكثر استقرارًا غالبًا.
- لا يحتاج Rollout loop وValue Model مثل PPO.

Lecture 6 — LLM Reasoning

Chain-of-Thought .1

بدل الانتقال مباشرة من السؤال إلى الإجابة:



Question → Answer

نجعل المودل ينتج خطوات وسطية:



Question → Reasoning Chain → Answer

هذا يساعد في:

- الرياضيات.
- المنطق.
- المسائل متعددة الخطوات.

Reasoning Models .2

الReasoning Model ليس مجرد مودل كتبنا له:



Think step by step

بل قد يتم تدريبه باستخدام Reinforcement Learning ليكتشف استراتيجيات Reasoning أفضل. مثلاً:



```
Question
↓
Generate several reasoning paths
↓
Check final answers
↓
Reward correct solutions
↓
Update model
```

Verifiable Rewards .3

في مسائل مثل الرياضيات والكود، يمكن التحقق من الإجابة آلياً. مثلاً:



```
Math answer == expected answer?
Code passes tests?
SQL query returns expected result?
```

هنا لا نحتاج دائماً إلى Human Preference أو Reward Model غير دقيق. يمكن استخدام:



```
Correct → reward = 1
Wrong → reward = 0
```

وهذه تسمى Verifiable Rewards.

GRPO مقابل PPO .4

PPO

غالبًا يحتاج:

- Policy Model
- Reference Model
- Reward Model
- Value Model
- Advantage estimation

GRPO

يولد مجموعة Responses لنفس السؤال:



$o_1, o_2, o_3, \dots, o_G$

ثم يحسب Reward لكل Response، ويقارنها بمتوسط المجموعة.

مثلاً:



Rewards = [0, 1, 0, 1]

Mean reward = 0.5

الإجابات التي Reward الخاصة بها أعلى من المجموعة تحصل على Positive Advantage، والعكس.

الميزة:

لا يحتاج إلى Value Model منفصل بالطريقة نفسها الموجودة في PPO.

لذلك GRPO أبسط وأقل استهلاكًا للـ Memory، وهو مناسب لتدريب Reasoning Models عندما نستطيع تقييم الإجابات.

Lecture 7 — Agentic LLMs

1. مشكلة معرفة LLM

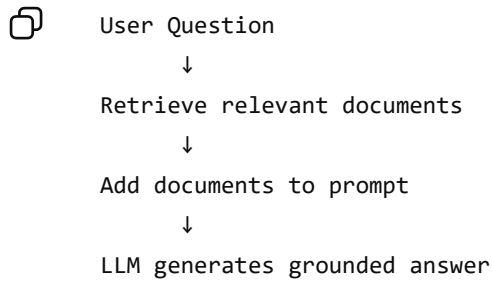
أوزان المودل ثابتة بعد التدريب.

هذا يعني أن المودل:

- قد لا يعرف معلومات حديثة.
 - لا يعرف بياناتك الخاصة.
 - قد يهلوس عندما تنقصه المعلومات.
-

RAG .2

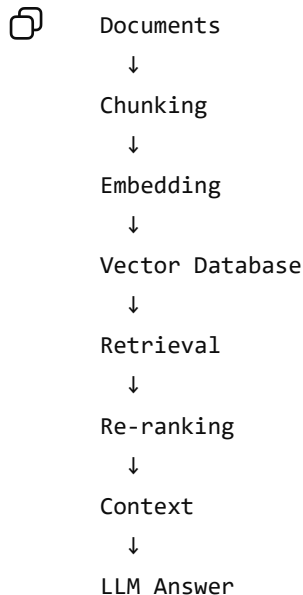
في Retrieval-Augmented Generation:



الـ RAG لا يغير أوزان المودل.

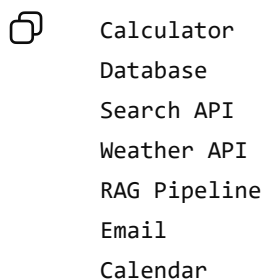
بل يمدّه بمعلومات خارجية وقت الـ Inference.

الـ Pipeline النموذجية:



Tool Calling .3

الـ LLM قد يقرر أنه يحتاج إلى Tool:



المودل لا ينفذ الـ Function داخليًا.

بل ينتج شيئًا مثل:

```
{  
  "tool": "get_weather",  
  "arguments": {  
    "city": "Cairo"  
  }  
}
```

الـ Application تنفذ الـ Tool، ثم تعيد النتيجة إلى المودل.

Agent .4

الـ Tool Calling قد يكون خطوة واحدة.

أما الـ Agent فهو Loop:



```
Question  
↓  
Reason  
↓  
Select tool  
↓  
Execute tool  
↓  
Observe result  
↓  
Reason again  
↓  
Another tool  
↓  
Final answer
```

هذا قريب من فكرة الـ ReAct:



Reasoning + Acting

فالـ Agent لا ينتج الإجابة مرة واحدة، بل يعمل عبر عدة خطوات.

Lecture 8 — LLM Evaluation

لماذا تقييم الـ LLMs صعب؟

لأن الإجابات ليست دائمًا Exact Match.

قد توجد إجابتان مختلفتان في الصياغة لكن كلاهما صحيحة.

لذلك نحتاج إلى عدة أنواع من التقييم.

1. LLM-as-a-Judge

نعطي مودلاً مقيماً:



Prompt

Model Response

Evaluation Criteria

ثم يرجع:



Rationale

Score

مثلاً:



Correctness: 8/10

Clarity: 9/10

Safety: 10/10

لكن الـ LLM Judge نفسه قد يكون متحيزاً أو غير دقيق، لذلك يجب اختباره ومعايرته.

2. مجالات التقييم

Knowledge

هل يستطيع المودل إعطاء حقائق صحيحة؟

مثال Benchmark:



MMLU

Reasoning

هل يستطيع حل مسائل متعددة الخطوات؟

أمثلة:



AIME

PIQA

Coding

هل يستطيع إنشاء Code صحيح وتشغيله أو إصلاح Repository؟

مثال:



SWE-bench

Safety

هل يرفض المحتوى الضار ويتبع السياسات؟

مثال:



HarmBench

الرسالة المهمة:

لا يوجد رقم واحد يصف جودة المودل بالكامل.

قد يكون المودل ممتازًا في Coding، ومتوسطًا في Safety أو Reasoning.

الجزء الثاني: Beyond Transformer-based LLMs

من الصفحة 43 تبدأ المحاضرة في تقديم الجزء الجديد:

هل يمكن استخدام Transformer في أشياء غير النصوص؟

الإجابة: نعم.

الTransformer بدأ أساسًا في Machine Translation، لكن الSelf-Attention فكرة عامة وليست مرتبطة باللغة فقط.

لماذا الTransformer قابل للاستخدام في مجالات مختلفة؟

لأنه يمتلك Weak Inductive Biases.

ما معنى Inductive Bias؟

هو الافتراض الذي تضعه الArchitecture عن شكل البيانات.

مثلاً، الـ CNN تفترض أن:

- البيانات لها Local patterns.
 - الـ Pixels القريبة أهم غالباً.
 - نفس الـ Filter يمكن تطبيقه في مواقع مختلفة.
- أما الـ Transformer فلا يفرض هذه الافتراضات بقوة.

هو يقول تقريباً:

أعطني مجموعة Elements، وسأتعلم العلاقات بينها باستخدام Attention.

هذه الـ Elements قد تكون:

- Text tokens
- Image patches
- Audio segments
- Video frames
- User-item interactions
- Molecules

ميزة الـ Weak Inductive Bias:

- Architecture أكثر عمومية.
- يمكنها تعلم علاقات متنوعة.

عيبها:

- قد تحتاج Compute و Data أكثر لتتعلم ما كانت Architecture أخرى تعرفه مسبقاً.

Vision Transformer — ViT

كيف نحول الصورة إلى Sequence؟

النص طبيعياً عبارة عن Tokens:



token1, token2, token3, ...

لكن الصورة عبارة عن Matrix:



Height × Width × Channels

الحل في ViT هو تقسيم الصورة إلى Patches.

مثلاً، صورة:



224 × 224 × 3

مع Patch size:



16 × 16

عدد Patches:

$$\frac{224}{16} \times \frac{224}{16} = 14 \times 14 = 196$$

إذن تتحول الصورة إلى Sequence من 196 Patch.

تحويل كل Patch إلى Embedding

كل Patch حجمه:



P × P × C

نقوم بعمل Flatten:



P × P × C → vector of size P²C

ثم نستخدم Linear Projection لتحويله إلى Dimension ثابت D :



Flattened patch



Linear Layer



Patch embedding of dimension D

مثلاً:



16 × 16 × 3 = 768

وقد يتم إسقاطه إلى Embedding بحجم 768 أيضًا أو حجم مختلف.

إضافة CLS Token

نضيف Token متعلمًا في بداية الـ Sequence:



[CLS], patch1, patch2, ..., patch196

هذا مشابه لـ BERT.

خلال الـ Self-Attention، يستطيع [CLS] جمع معلومات من جميع الـ Patches.

بعد آخر Encoder Layer نأخذ تمثيل [CLS] ونرسله إلى الـ Classification Head:



CLS embedding



MLP / Linear Head



Class probabilities

مثلاً:



teddy bear: 0.92

dog: 0.03

cat: 0.01

ViT Positional Embeddings في ViT

الـ Transformer لا يعرف الموقع، لذلك نضيف الـ Position Embedding لكل Patch.

وإلا فلن يعرف الفرق بين:

- Patch في أعلى الصورة.
- Patch في أسفل الصورة.
- Patch يمثل الوجه.
- Patch يمثل الكتاب.

إذن:



Patch embedding + Position embedding

ثم تدخل الـ Sequence إلى الـ Transformer Encoder.

ViT Pipeline كاملة



Image



Split into patches



Flatten each patch



```
Linear projection
↓
Add [CLS]
↓
Add position embeddings
↓
Transformer Encoder
↓
Take final [CLS] embedding
↓
Classification Head
↓
Class: teddy bear
```

فكرة ViT الأساسية:

نتعامل مع Patches الصورة كما نتعامل مع Tokens النص.

VLM — Vision Language Model

ViT وحده يستطيع فهم الصور وتصنيفها.

لكننا نريد مودلاً يأخذ:



Image + Text Question

ويرجع:



Text Answer

مثلاً:



Image: teddy bear reading
Question: How cute is this teddy bear?
Answer: Very cute!

هذا يسمى Vision Language Model.

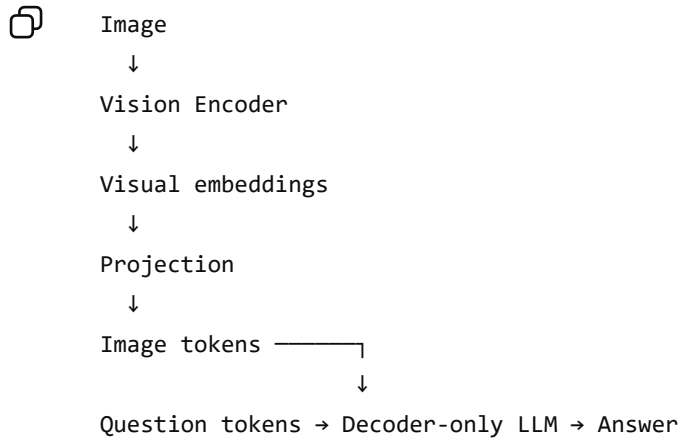
المحاضرة تعرض طريقتين أساسيتين لربط الصورة بالـ Language Model.

Method 1 — Recycle Decoder-only Architecture

الفكرة:

1. نمرر الصورة إلى Vision Encoder.

2. نحصل على Image Embeddings.
3. نستخدم **Projection Layer** لتحويلها إلى نفس Dimension الذي يفهمه الـ LLM.
4. نتعامل معها كأنها Tokens إضافية.
5. ندخل Image Tokens و Text Tokens معًا إلى Decoder-only LLM.



المودل يرى Sequence شبيهًا بـ:

```
<image_token_1>
<image_token_2>
...
<question>
How cute is this teddy bear?
```

ثم يولد الإجابة Autoregressively.

هذه الطريقة تحافظ على أغلب بنية الـ LLM كما هي، ولا تحتاج إلى إعادة تصميم كاملة للمودل.

Method 2 — Cross-Attention

في هذه الطريقة تبقى الصورة في مسار منفصل:

```
Image -> Vision Encoder -> Image representations
```

والنص يدخل الـ Decoder.

داخل الـ Decoder توجد Cross-Attention Layers:

- الـ Queries تأتي من Text tokens.
- الـ Keys والـ Values تأتي من Image representations.

```
Q = text representation
K, V = image representation
```

عندما يريد المودل توليد كلمة مرتبطة بالصورة، ينظر إلى الـ Visual Features عبر Cross-Attention.



Text Decoder

└ Self-Attention over text

└ Cross-Attention over image features

الفرق بين الطريقتين

الفكرة

الطريقة

نحول الصورة إلى Tokens وندخلها مباشرة مع النص

Image as tokens

نحافظ على image representations منفصلة ويتفاعل النص معها عبر Cross-Attention

Cross-Attention

Image tokens

- أبسط في الدمج مع Decoder-only LLM.
- Architecture موحدة.
- لكن الصورة قد تستهلك عددًا كبيرًا من Tokens.

Cross-Attention

- فصل أوضح بين Modalities.
 - تحكم أكبر في طريقة قراءة الصورة.
 - لكنه يحتاج تعديلات داخل بنية الـ LLM.
- المحاضرة تعرض أيضًا إمكانية وجود مسارات منفصلة لـ:

- Image
- Video
- Speech
- Text

ثم يتم ربطها بالـ Language Model.

استخدامات Transformer خارج النص

المحاضرة تلخص أن الـ Transformers تستخدم في:



Text generation → LLMs

Image understanding → ViT

Image generation → DiT / MM-DiT

Video understanding

Speech
Recommendation systems
Multimodal systems

وهنا الرسالة المهمة:

Transformer لم يعد مجرد Language Architecture؛ بل أصبح General-purpose sequence architecture.

الجزء الثالث: Diffusion LLMs

هذا هو أهم موضوع جديد في المحاضرة.

أولاً: مشكلة Autoregressive LLMs

الـ LLM التقليدي يولد Token واحدًا في كل مرة:

[BOS]
↓
A
↓
teddy
↓
bear
↓
is
↓
reading

لا يمكن توليد reading قبل معرفة is ، ولا يمكن توليد is قبل معرفة bear .

لذلك الـ Inference Sequential:

token 1 → token 2 → token 3 → token 4

إذا كان المطلوب توليد 1000 Token، نحتاج تقريبًا إلى 1000 Decoding Steps.

لماذا التدريب Parallel لكن الـ Inference غير Parallel؟

أثناء التدريب، لدينا الجملة الصحيحة كاملة:

A teddy bear is reading

ونستخدم Teacher Forcing.

يدخل المودل:



[BOS] A teddy bear is

ويتوقع في الوقت نفسه:



A teddy bear is reading

بفضل الـ Causal Mask يمكن حساب Loss لكل المواقع في Forward Pass واحد.

أما أثناء الـ Inference فلا نعرف الـ Token التالي، ولذلك يجب:

1. توقع Token.
2. إضافته إلى Context.
3. تشغيل المودل مرة أخرى.
4. توقع Token آخر.

إذن:



Training → parallel across positions
Inference → sequential across generated tokens

فكرة Diffusion في الصور

قبل النص، تشرح المحاضرة Image Diffusion.

Forward Process

نبدأ بصورة حقيقية:



$x_0 = \text{clean image}$

ثم نضيف Noise تدريجيًا:



$x_0 \rightarrow x_1 \rightarrow x_2 \rightarrow \dots \rightarrow x_T$

حتى تصبح الصورة في النهاية Noise تقريبًا.

هذه العملية محددة رياضياً ولا نحتاج إلى تعلمها.

Reverse Process

ندرب مودلاً على عكس العملية:



noise → less noise → clearer image → final image

أي يتعلم:

$$p_{\theta}(x_{t-1} | x_t)$$

عند التوليد:

1. نبدأ من Random Noise.
2. يزيل المودل جزءاً من Noise.
3. نكرر العملية عدة مرات.
4. تظهر الصورة النهائية.

تشبيه المحاضرة هو النحت:

التمثال موجود داخل كتلة الرخام، والنحات يزيل الأجزاء غير المطلوبة تدريجيًا.

كيف ننقل Diffusion من الصور إلى النص؟

في الصور، Noise عبارة عن قيم Pixel عشوائية.

أما النص Discrete، فلا يمكن إضافة Gaussian Noise إلى كلمة مثل:



teddy + 0.3 noise

لذلك نستخدم Token خاصًا:



[MASK]

الMASK في Text Diffusion يلعب دور الNoise.

Forward Process للنص

نبدأ بجملته:



A teddy bear is reading

ثم نخفي بعض Tokens:



A [MASK] bear is [MASK]

ثم نخفي المزيد:



[MASK] [MASK] bear [MASK] [MASK]

حتى نصل في النهاية إلى:



[MASK] [MASK] [MASK] [MASK] [MASK]

Reverse Process للنص

ندرب المودل على استعادة Tokens تدريجيًا:



[MASK] [MASK] [MASK] [MASK] [MASK]

↓

A [MASK] bear is [MASK]

↓

A teddy bear is reading

هذا يسمى:



MDM = Masked Diffusion Model

كيف يتم التوليد في Diffusion LLM؟

بدل البدء بـ [BOS] وتوليد Token واحد:



[BOS] → A → teddy → bear → ...

نبدأ بعدد من Masks:



[MASK] [MASK] [MASK] [MASK] [MASK]

في كل Forward Pass المودل توقع عدة مواقع معًا.

مثلاً:

Step 1



[MASK] [MASK] bear [MASK] reading

Step 2



A [MASK] bear is reading

Step 3



A teddy bear is reading

وهكذا قد نحصل على خمسة Tokens في ثلاث Forward Passes بدل خمس Passes منفصلة.

الفكرة:

الـ Diffusion LLM يمكنه تحديث عدة Tokens بالتوازي.

الفرق بين BERT و Diffusion LLM

قد يبدو لك أن هذا هو نفسه Masked Language Modeling في BERT، لكن توجد فروق مهمة.

BERT

أثناء التدريب:



A [MASK] bear is reading

ويحاول توقع الـ MASK.

لكن BERT في الأساس:

- Encoder-only.
- مصمم لفهم النص.
- ليس مصممًا عادة لتوليد جملة كاملة من Masks.
- يقوم غالبًا بعملية Mask prediction واحدة.

Diffusion LLM

- مصمم كـ Generative Model.
- قد يبدأ بجملة كلها Masks.
- ينفذ عدة Denoising Steps.
- يمكنه تعديل عدة مواقع في كل خطوة.
- يتعلم عملية انتقال من Highly corrupted text إلى Clean text.

إذن:



BERT MLM → representation learning / understanding

Diffusion LLM → iterative text generation

لماذا قد تكون Diffusion LLMs أسرع؟

في Autoregressive Model:



100 tokens → 100 تقريبًا sequential decoding steps

في Diffusion Model قد يتم إظهار عدة Tokens في كل Step:



100 tokens → عدد أقل من denoising steps

المحاضرة تشير إلى إمكانية الوصول في بعض الأنظمة أو الإعدادات إلى معدل Output Tokens أعلى كثيرًا، وتعرض رقمًا يصل إلى نحو 10x مقارنة بالAutoregressive Modeling.

لكن هذا لا يعني أن كل Diffusion LLM أسرع عشر مرات دائمًا؛ فالسرعة تعتمد على:

- عدد Denoising Steps.
- حجم المودل.
- الـ Hardware.
- طول الـ Sequence.
- عدد Tokens التي يتم تثبيتها في كل خطوة.
- كفاءة التنفيذ.

ما المهام التي قد تناسب Diffusion LLMs؟

لأن المودل يرى عدة مواقع ويحدثها معًا، فقد يكون مناسبًا لمهام مثل:

- Text infilling
 - Code editing
 - تصحيح جملة كاملة.
 - إعادة كتابة أجزاء متعددة.
 - ملء Templates.
 - Structured generation
 - تعديل Output موجود بدل توليده من اليسار إلى اليمين.
- في Autoregressive generation، القرارات المبكرة يصعب تعديلها لاحقًا.
- أما Diffusion فيمكنه نظريًا مراجعة أجزاء متعددة أثناء عملية الـ Denoising.

تحديات Diffusion LLMs

المحاضرة تذكر تحديين أساسيين.

1. Performance

الـ Autoregressive LLMs ناضجة جدًا وتم تدريبها على أحجام هائلة.

لذلك يجب على Diffusion LLMs أن تثبت قدرتها على المنافسة في:

- جودة النص.
- Coherence.
- Reasoning.
- Coding.
- Long-context.
- Instruction following.
- Safety.

السرعة وحدها لا تكفي.

2. Adapt ARM-based Techniques

خلال السنوات الماضية، تم تطوير منظومة ضخمة حول الـ Autoregressive Models:

- KV Cache.
- Speculative Decoding.
- Streaming.
- RLHF.
- DPO.
- Tool Calling.
- Reasoning RL.
- Serving and batching.
- Quantization.
- Prefix caching.

بعض هذه التقنيات مبني على افتراض:



Generate one next token at a time

لذلك لا يمكن نقلها مباشرة إلى Diffusion LLM، بل يجب إعادة تصميمها أو تكييفها.

مقارنة Diffusion LLM و Autoregressive

Diffusion LLM	Autoregressive LLM	الجانِب
Masks مجموعة	Prompt أو [BOS]	البداية
Tokens تحديث عدة	Token بعد Token	التوليد
غير مقيد تمامًا بالاتجاه	Left-to-right غالبًا	الاتجاه
أكثر قابلية للـParallelism	Sequential	Inference
مجال أحدث	Mature جدًا	النضج
أصعب نسبيًا	طبيعي	Streaming
Denoising ممكن أثناء	محدود	تعديل Tokens سابقة
قد يكون أقل	Tokens قريب من عدد	عدد Forward Passes

الجزء الرابع: Closing Thoughts

1. Cross-pollination Between Modalities

المحاضرة توضح أن مجالات AI أصبحت تتبادل الأفكار.

فكرة من الصور إلى النص



Diffusion → Diffusion LLM

الـDiffusion اشتهر في Image Generation، ثم بدأ استخدامه في Text Generation.

فكرة من النص إلى الصور



Transformer → Vision Transformer / Diffusion Transformer

الـTransformer بدأ مع النص والترجمة، ثم انتقل إلى الصور والفيديو.

2. Diffusion Transformer — DiT

Image Diffusion Models القديمة استخدمت غالبًا U-Net.

أما DiT فيستخدم Transformer لمعالجة Noisy Latent Patches.



Noisy latent image



Patchify



Transformer blocks



Predict noise / denoising direction

إذن يمكن الجمع بين:



Diffusion training objective

+

Transformer architecture

الـ Diffusion يحدد طريقة التوليد، بينما الـ Transformer هو الـ Network الذي ينفذ عملية الـ Denoising.

وهذه نقطة مهمة:

Diffusion و Transformer ليسا بالضرورة بديلين لبعضهما.

قد يكون لديك:



Transformer-based Diffusion Model

مثل الـ DiT للصورة، أو الـ Transformer-based Masked Diffusion Model للنص.

3. Input Representation

المحاضرة تشير إلى أن طريقة تمثيل الـ Input ما زالت موضوعًا بحثيًا مهمًا.

في النص لدينا Tokens.

في الصور لدينا:

- Patches.
- Visual tokens.
- Compressed representations.
- OCR representations.

أحد الاتجاهات هو ضغط Context بصريًا؛ فبدل تمثيل صفحة طويلة بآلاف الـ Text Tokens، قد تمثل كصورة أو مجموعة الـ Visual Tokens مضغوطة ثم يفهمها الـ Multimodal Model.

الرسالة هنا:

الـTokenizer أو Input Encoder قد يكون بنفس أهمية الـModel Architecture.

4. Multimodality للـPositional Encoding

في النص نحتاج غالبًا إلى بعد واحد:



position 1, 2, 3, 4

لكن الصورة لها بعدان:



height
width

والفيديو قد يضيف بعدًا ثالثًا:



time

لذلك ظهرت Variants من RoPE تأخذ في الاعتبار:

- Horizontal position
- Vertical position
- Temporal position
- ترتيب Text tokens

مثل Multimodal Scalable RoPE.

البحث الأساسي ما زال مستمرًا

رغم التقدم، لا يوجد اتفاق نهائي على أفضل تصميم.

1. Optimizers

ما الخيار الأفضل؟



AdamW
Muon-like optimizers
Other optimizer variants

الـOptimizer قد يؤثر في:

- Stability
- سرعة التقارب.
- مقدار Memory.

- قدرة Scaling.

2. Normalization

توجد أسئلة حول:

- LayerNorm أم RMSNorm؟
- Pre-Norm أم Post-Norm؟
- أين نضع الـ Normalization؟
- كيف نمنع انفجار أو اختفاء Activations؟

3. Attention Design

لا يزال الاختيار بين:



MHA
MQA
GQA
Other efficient attention mechanisms

يعتمد على التوازن بين:

- .Quality
- .Memory
- .Inference speed
- .Training stability

4. Activation Functions

مثل:



ReLU
GELU
SwiGLU
Other gated activations

اختيار الـ Activation يؤثر في كفاءة وقدرة الـ Feed-Forward Network.

5. MoE أم Dense؟

Dense Model

كل Parameters المستخدمة في كل Token.

MoE Model

عدد Parameters كبير، لكن يتم تنشيط جزء منها فقط.

السؤال ليس أن أحدهما أفضل دائمًا؛ بل يعتمد على:

- Infrastructure.
- Training budget.
- Inference requirements.
- Communication cost.
- Target model size.

6. عدد الـ Layers وأبعاد المودل

حتى مع Budget ثابت يمكن توزيعه بطرق مختلفة:



More layers + narrower hidden size
Fewer layers + wider hidden size
More attention heads
Larger FFN

ولا توجد إجابة واحدة مثالية لكل الحالات.

مشكلة الـ Data — الـ وقود

الـ Data هي وقود الـ LLMs.

لكن كمية النص البشري عالي الجودة محدودة.

مع انتشار المحتوى المولد بالذكاء الاصطناعي، قد يحدث:



Models generate data
↓
New models train on generated data
↓
Generated errors and biases accumulate

وهذا قد يؤدي إلى تدهور يسمى أحيانًا:



Model collapse

لذلك من أهم أسئلة المستقبل:

- من أين نحصل على High-quality data؟
- كيف ننقي Web data؟
- كيف نتحقق من Synthetic data؟
- كيف نمنع تكرار أخطاء المودلات؟
- كيف نستفيد من Human feedback بكفاءة؟

Synthetic Data ليست سيئة في حد ذاتها، لكنها تحتاج إلى:

- Filtering.
- Verification.
- Diversity.
- Ground truth.
- تجنب الاعتماد الدائري غير المنضبط.

هل الـ Transformer هو أفضل Architecture نهائياً؟

المحاضرة تطرح سؤالاً مهماً:

هل الـ Transformer أفضل Architecture ممكنة، أم أنه فقط أفضل ما لدينا حالياً؟

نجاح الـ Transformer لا يعني أنه نهاية البحث.

توجد مشكلات مثل:

- Quadratic Attention cost مع طول الـ Sequence.
- KV Cache الكبير.
- Memory movement.
- Autoregressive bottleneck.
- الحاجة إلى Data و Compute ضخمين.

قد تظهر Architectures جديدة أو Hybrid Systems تجمع بين:

- Transformer.
- State Space Models.
- Recurrence.
- Retrieval.
- External memory.
- Diffusion.
- Specialized modules.

المستقبل قد لا يكون Architecture واحدة، بل مجموعة مكونات تعمل معاً.

من أعلى Performance إلى أفضل Quality/Cost

في السابق كان التركيز:



ما أقوى مودل؟

لكن في المنتجات الحقيقية، السؤال الأهم غالبًا:



ما أفضل جودة يمكن الحصول عليها مقابل التكلفة؟

قد يكون مودل صغير أرخص بمئة مرة وأقل جودة بنسبة بسيطة هو الخيار الأفضل.

لذلك نقارن:



Quality
Latency
Price
Energy
Memory
Throughput
Reliability

هذا يسمى التفكير في **Pareto Frontier**.

المودل الموجود على الـ Pareto Frontier يعني أنه لا يمكن تحسين جانب مثل الجودة دون التضحية بالتكلفة، أو تقليل التكلفة دون التضحية بالجودة.

في Production لا تختار دائمًا أقوى مودل، بل تختار المودل الذي يحقق أفضل Trade-off لتطبيقك.

Hardware Optimization

المشكلة

الـ GPUs ممتازة في:



Matrix-Matrix Multiplication
Matrix-Vector Multiplication

لكن تكلفة تشغيل الـ Transformer ليست كلها Compute.

خاصة أثناء Decoding، جزء كبير من التكلفة يأتي من:



Reading and writing KV Cache
Moving data between memory and compute units

أحيانًا تكون العملية **Memory-bound** وليست **Compute-bound**.

أي أن الـ GPU ينتظر وصول البيانات بدل أن يكون مشغولًا بالحساب.

KV Cache

لكل Token سابق نحفظ الـ Keys والـ Values في كل Attention Layer.

كلما زاد:

- Context length
- Batch size
- Number of layers
- Number of KV heads
- Head dimension

زاد حجم الـ KV Cache.

وهذا أحد أسباب استخدام الـ MQA و GQA.

Hardware-Software Co-design

الفكرة المستقبلية هي عدم تصميم الـ Model وحده ثم محاولة تشغيله على الـ Hardware عام.

بل تصميم الاثنين معًا:



Model architecture



Hardware architecture

المحاضرة تعرض اتجاهًا لدعم الـ Attention مباشرة داخل الـ Hardware:

- تخزين الـ KV Cache في الـ Dedicated cells.
- تنفيذ بعض العمليات باستخدام الـ Analog signals.
- تقليل نقل البيانات.
- تقريب الـ Memory من الـ Compute.

الدراسة المشار إليها في الشرائح تعرض في إعدادها التجريبي وفورات كبيرة جدًا في الـ Latency والطاقة مقارنة بـ H100، لكن هذه النتائج لا تعني بالضرورة أن كل الـ Workload أو نظام الـ Production سيحصل على الأرقام نفسها.

المغزى:

قد تأتي القفزة القادمة من الـ Hardware بقدر ما تأتي من تحسين الـ Model.

تأثير الـ LLMs في الحياة اليومية

المحاضرة ترى أن تأثير الـ LLMs بدأ بالفعل في:

- Coding.
- Text-to-query و Text-to-SQL.
- Conversational assistants.
- Web search.
- Creativity.
- Learning.
- كتابة وتحليل المستندات.

لكن الاستخدامات القادمة قد تكون أعمق.

الاتجاه القريب: Democratization of Agents

سيتمكن المستخدم العادي من إنشاء Automations و Agents دون برمجة معقدة.

مثلاً:



Read my emails
Find invoices
Update spreadsheet
Create weekly report
Send it to manager

بدل كتابة Workflow برمجي كامل، يصف المستخدم الهدف باللغة الطبيعية.

Browser-level Agents

قد يعمل الـ Agent داخل المتصفح ليقوم بـ:

- التنقل بين المواقع.
- ملء Forms.
- جمع المعلومات.
- مقارنة المنتجات.
- تنفيذ حجوزات.
- التعامل مع Web applications.

OS-level LLM

الخطوة الأبعد هي أن يصبح الـ LLM طبقة داخل نظام التشغيل.

بدل فتح كل Application وتنفيذ الخطوات يدويًا:



Organize these files
Extract tables from PDFs
Email the summary
Schedule a meeting

يقوم Agent بالتعامل مع التطبيقات والملفات والخدمات المختلفة.

Autonomous Agents

الرؤية طويلة المدى:



High-level goal
↓
Agent creates plan
↓
Uses many tools
↓
Monitors results
↓
Corrects errors
↓
Completes task

لكن هذا ما زال صعبًا بسبب:

- تراكم الأخطاء عبر الخطوات.
- ضعف التخطيط طويل المدى.
- Tool failures.
- Security.
- Permissions.
- Cost.
- عدم موثوقية المودل.
- صعوبة معرفة متى يجب أن يتوقف أو يطلب مساعدة.

التحديات المستمرة

1. Fixed Weights

بعد انتهاء التدريب، أوزان المودل لا تتغير أثناء الاستخدام العادي.

لذلك لا يمتلك Continuous Learning حقيقيًا.

يمكن استخدام:

- RAG.
- Memory systems.
- Periodic fine-tuning.

لكنها ليست نفس أن يتعلم المودل باستمرار وبأمان من كل تفاعل.

التعلم المستمر قد يسبب أيضًا:

- Catastrophic forgetting.
- تعلم معلومات خاطئة.
- Data poisoning.
- Privacy problems.

2. Hallucinations

المودل يتعلم:



Generate likely text

وليس:



Guarantee factual truth

لذلك قد ينتج إجابة مقنعة لغويًا لكنها خاطئة.

الحلول تشمل:

- RAG.
- Tool use.
- Verifiers.
- Citations.
- Constrained generation.
- Human review.
- Better evaluation.

لكن المشكلة لم تختفِ تمامًا.

3. Personalization

نريد أن يعرف النظام:

- تفضيلات المستخدم.
- أسلوبه.
- مشروعاته.
- أهدافه.
- سياقه السابق.

لكن يجب التوازن بين Personalization و:

- Privacy.
- Security.
- عدم تخزين معلومات حساسة بلا داع.
- التحكم في ما يتم تذكره أو نسيانه.

4. Interpretability

ما زلنا لا نفهم بصورة كاملة:

- لماذا اتخذ المودل قرارًا معينًا؟
- أين تُخزن معلومة داخل الأوزان؟
- ماذا يمثل Neuron أو Attention Head؟
- هل الـ Reasoning حقيقي أم مجرد Pattern generation؟
- كيف نكتشف سلوكًا خطيرًا قبل ظهوره؟

5. Safety

كلما أصبح المودل أكثر Agentic، لم تعد المشكلة مجرد نص سيئ.

قد يمتلك صلاحيات لـ:

- إرسال Email.
- تعديل Database.
- شراء منتجات.
- تشغيل Code.
- حذف ملفات.

إذن يجب بناء:

- Permission systems.
- Sandboxing.
- Human approval.
- Audit logs.
- Tool restrictions.
- Monitoring.

كيف تظل متابعًا للمجال بعد الكورس؟

Papers

تابع:



arXiv → Computer Science → Computation and Language

والConferences:



General ML:

NeurIPS

ICML

ICLR

NLP:

ACL

EMNLP

NAACL

Code

عند قراءة Paper ابحث عن:

- Official GitHub repository
- Training code
- Model checkpoints
- Dataset
- Evaluation scripts

وتابع:



Hugging Face trending papers

Hugging Face models and datasets

لا تكتفِ بقراءة الPaper؛ تشغيل الCode يكشف تفاصيل لا تظهر في النص.

مصادر عملية ونظرية

المحاضرة تقترح متابعة:

- الباحثين ومهندسي الصناعة.
 - Technical blogs للشركات والمختبرات.
 - قنوات شرح الأوراق.
 - قنوات تطبيقية مثل Hugging Face و Karpathy ومصادر المطورين.
- لكن الأهم هو عدم الاعتماد على أخبار Social Media وحدها.

أفضل تسلسل:



Announcement



Technical report / paper



Code



Independent evaluation



Your own experiment

ما الذي يجب أن تخرج به من الكورس؟

المستوى الأول: فهم المودل

يجب أن تكون قادرًا على شرح:

- .Tokenization
- .Embeddings
- .Attention
- .Q, K, V
- .Multi-Head Attention
- .Transformer Blocks
- .Encoder vs Decoder
- .Autoregressive generation

المستوى الثاني: فهم التدريب

يجب أن تميز بين:



Pretraining

Supervised Fine-Tuning

Preference Tuning

RLHF

وأن تعرف في أي مرحلة يتم تعديل الأوزان ولماذا.

المستوى الثالث: فهم النظام

يجب أن تفهم أن تطبيق LLM حقيقي قد يحتوي على:



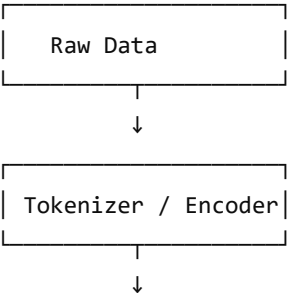
- LLM
- RAG
- Vector Database
- Re-ranker
- Tools
- Agent loop
- Memory
- Guardrails
- Evaluation
- Monitoring

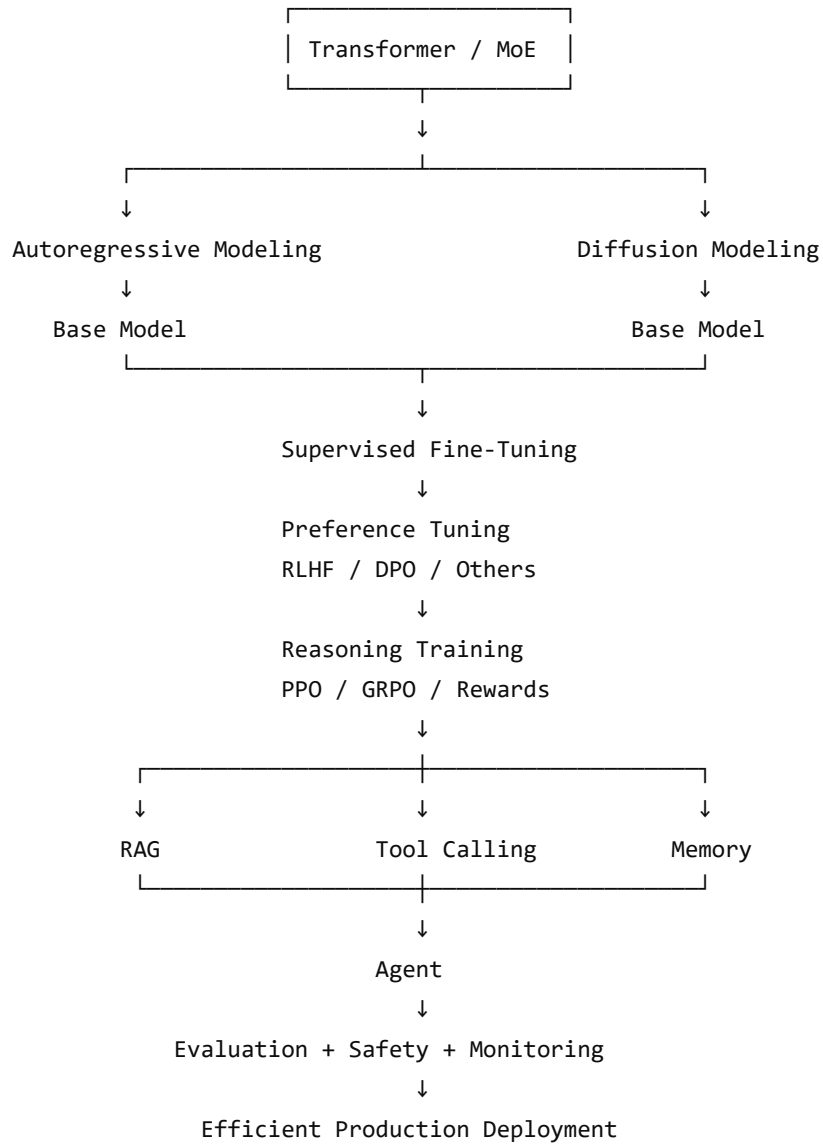
المستوى الرابع: فهم الـ Production

النظام الجيد ليس الأقوى فقط، بل يجب أن يكون:

- سريعًا.
- رخيصًا.
- قابلًا للمراقبة.
- آمنًا.
- قابلًا للتقييم.
- قادرًا على التعامل مع الفشل.
- قابلًا لتبديل المودلات أو قواعد البيانات أو الأدوات.

الخريطة النهائية للمجال





الخلاصة النهائية للمحاضرة

المحاضرة لا تقول إنك انتهيت من تعلم الـ LLMs، بل تقول إنك أصبحت تملك الأساس الذي يسمح لك بفهم القادم.

أهم خمس رسائل فيها:

1. الـ Transformer فكرة عامة وليست خاصة بالنص؛ يمكنها التعامل مع Images و Audio و Video وغيرها.
2. الـ Autoregressive generation ليست الطريقة الوحيدة لتوليد النص؛ Diffusion LLMs تحاول توليد أو تعديل عدة Tokens بالتوازي.
3. المستقبل Multimodal و Agentic؛ المودل سيتعامل مع الصور والصوت والأدوات والبرامج، وليس النص فقط.
4. الجودة وحدها ليست الهدف؛ التكلفة والسرعة والطاقة والموثوقية مهمة بنفس القدر في Production.